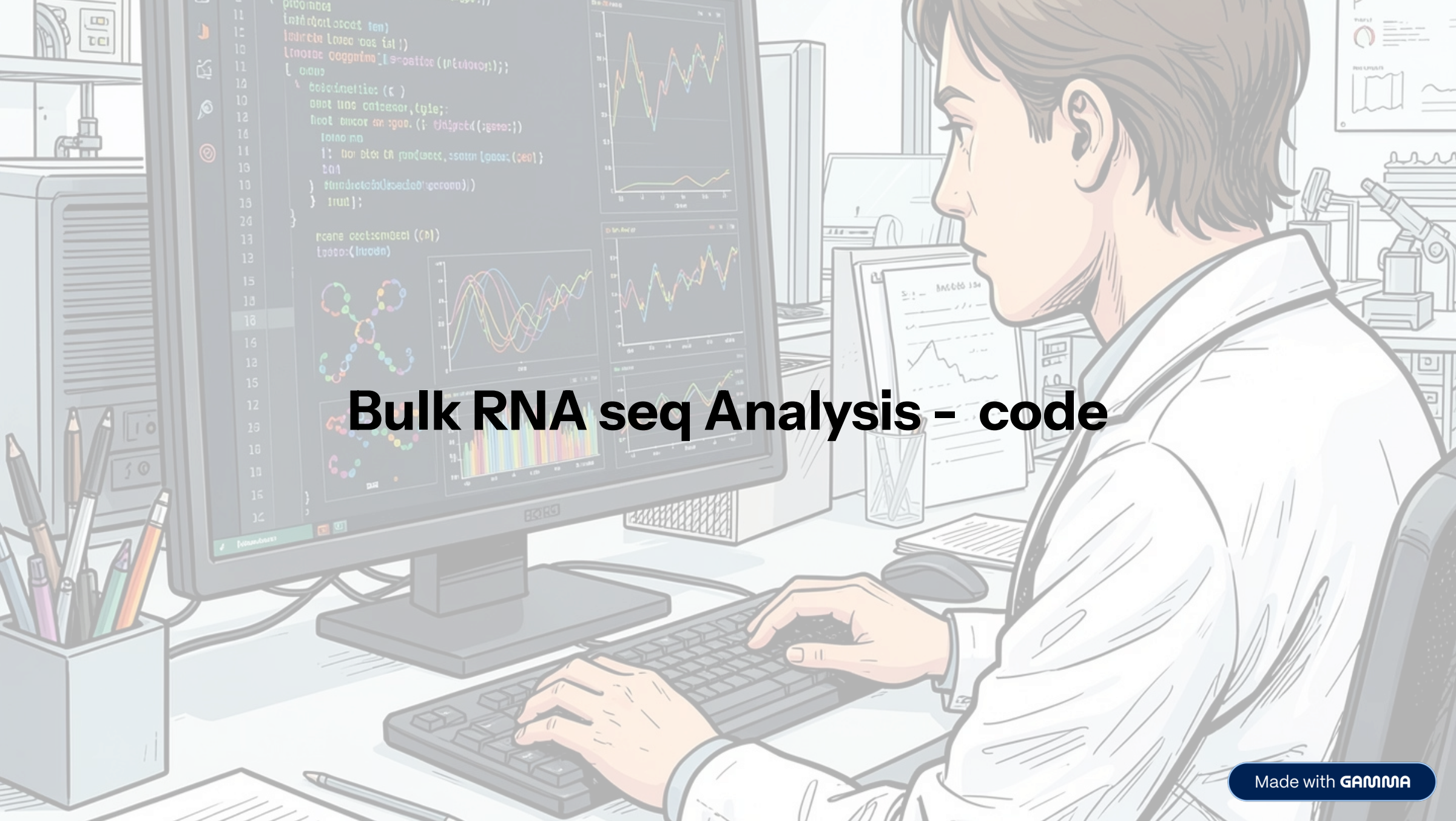

Steps of bulk RNA-seq analysis

- Quality control of raw sequencing data (FastQC)
- Read trimming and filtering (Trimmomatic)
- Alignment to reference genome (STAR)
- Gene-level quantification (HTSeq)
- Differential expression analysis (DESeq2)
- Functional analysis (GO)

Bulk RNA-seq analysis can be performed on a local computer (PC) or on an HPC cluster

- **Choice depends on:**
- **dataset size (number of samples, read depth)**
- **computational resources (CPU, RAM, storage)**

How does it perform in HPC (High Performance Cluster)

- Log in to the HPC cluster and organize raw RNA-seq data into structured project folders.
- Load required bioinformatics software using modules or conda environments.
- Write a Slurm job script specifying CPUs, memory, time, and analysis commands.
- Submit the job using `sbatch`, where the scheduler runs it on compute nodes.
- Collect output files for downstream analysis such as differential expression and pathway analysis.

following is the general format of a slurm script .

#!/bin/bash

################################

Slurm job configuration

################################

#SBATCH --job-name=RNAseq_job # Job name #SBATCH --output=job_%j.out # Standard output (%j = job ID) #SBATCH --error=job_%j.err # Standard error file #SBATCH --time=24:00:00 # Time limit (hh:mm:ss) #SBATCH --cpus-per-task=8 # Number of CPU cores #SBATCH --mem=32G # Memory required #SBATCH --partition=standard # Queue/partition name #SBATCH --mail-type=END,FAIL # Email notifications (optional) #SBATCH --mail-user=**your_email@domain.com**

################################

Load required modules

################################

module load fastqc module load star module load samtools module load R

OR activate conda environment

source activate rnaseq_env

################################

Set working directory

################################

cd /path/to/your/project/

################################

Run analysis commands

################################

Example: FastQC

fastqc raw_data/*.fastq -o qc_results/

Example: Alignment (STAR)

STAR --runThreadN 8
--genomeDir genome_index/
--readFilesIn raw_data/sample.fastq
--outFileNamePrefix results/sample_

Example: FeatureCounts

featureCounts -T 8 -a annotation.gtf -o counts.txt aligned.bam

################################

End of script

################################

echo "Job completed successfully"

Following is the **workflow** showing how each RNA-seq step is done on a **normal PC vs HPC using Slurm scripts.**

Here's a complete, clean **step-by-step setup for running FastQC on a Mac**

FASTQC Setup on Mac (Full Guide)

1. Install Homebrew (package manager)

Homebrew makes installing bioinformatics tools easy.

Open Terminal and run:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

After install, confirm:

```
brew --version
```

2. Install Java (required for FastQC)

FastQC runs on Java.

```
brew install openjdk
```

Then link Java (important step on Mac):

```
sudo ln -sfn /opt/homebrew/opt/openjdk/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk.jdk
```

Check Java:

```
java -version
```

3. Install FastQC

Option A (recommended): Homebrew

```
brew install fastqc
```

Check:

```
fastqc --version
```

Option B (manual install)

If brew fails:

1. Download: **https://www.bioinformatics.babraham.ac.uk/projects/fastqc/**
2. Unzip:

```
unzip fastqc_v*.zip
```

1. Make executable:

```
chmod +x FastQC/fastqc
```

1. Run:

```
./FastQC/fastqc sample.fastq
```

4. Install MultiQC (recommended upgrade step)

MultiQC combines all FastQC reports into one dashboard.

Install via Homebrew:

```
brew install multiqc
```

Check:

```
multiqc --version
```

5. Run FastQC (your first QC)

Go to your folder:

```
cd /path/to/your/files
```

Run:

```
fastqc *.fastq
```

Output:

- `.html` report (open in browser)
- `.zip` results file

6. View results

Open HTML report:

```
open sample_fastqc.html
```

7. (Optional but powerful) Run MultiQC

After multiple samples:

```
multiqc .
```

Then open:

```
open multiqc_report.html
```

Here is a SLURM script for FastQC

```
#!/bin/bash
#SBATCH --job-name=fastqc_job          # Job name
#SBATCH --output=fastqc_%j.out         # Output log
#SBATCH --error=fastqc_%j.err          # Error log
#SBATCH --ntasks=1                     # Single task
#SBATCH --cpus-per-task=4              # CPU cores
#SBATCH --time=02:00:00                # Time limit
#SBATCH --mem=8G                       # Memory

############################################
# 1. LOAD MODULES
############################################

module load java

############################################
# 2. USER-DEFINED PATHS (EDIT ONLY THIS SECTION)
############################################

FASTQC_PATH="/path/to/FastQC/fastqc"
INPUT_DIR="/path/to/your/FASTQ_files"
OUTPUT_DIR="$INPUT_DIR/fastqc_results"

############################################
# 3. CREATE OUTPUT DIRECTORY
############################################

mkdir -p "$OUTPUT_DIR"

############################################
# 4. LOG FILE
############################################

LOG_FILE="$OUTPUT_DIR/fastqc_log.txt"
echo "FastQC started at $(date)" > "$LOG_FILE"

############################################
# 5. RUN FASTQC (ALL FASTQ FILES)
############################################

for FASTQ_FILE in "$INPUT_DIR"/*.fastq*; do
  if [ -f "$FASTQ_FILE" ]; then
    echo "Processing: $FASTQ_FILE" >> "$LOG_FILE"

    "$FASTQC_PATH" "$FASTQ_FILE" -o "$OUTPUT_DIR" -t $SLURM_CPUS_PER_TASK
  else
    echo "No FASTQ files found matching pattern" >> "$LOG_FILE"
  fi
done

############################################
# 6. FINISH LOG
############################################

echo "FastQC completed at $(date)" >> "$LOG_FILE"
```

Key Improvements vs your version

✔ Works for BOTH:

- single-end (`.fastq`)
- paired-end (`_R1 / _R2 / .fastq.gz`)

✔ Cleaner logic:

Instead of:

```
*_R1.fastq *_R2.fastq
```

It safely processes:

```
*.fastq*
```

✔ Better SLURM usage:

Uses:

```
-t $SLURM_CPUS_PER_TASK
```

So CPU allocation is dynamic.

✔ Better logging:

Separate:

- `.out` → SLURM system output
- `.err` → errors
- `fastqc_log.txt` → QC tracking

✔ More portable:

You only edit:

```
FASTQC_PATH
INPUT_DIR
```

step-by-step workflow to run your Trimmomatic job on a normal PC

Step 1: Install Java (required)

Check if Java is installed:

```
java -version
```

If not installed:

- Windows: install Temurin / Oracle JDK
- Linux:

```
sudo apt update
sudo apt install openjdk-21-jdk
```

- macOS:

```
brew install openjdk
```

Step 2: Download Trimmomatic

Download:

Trimmomatic

You should have:

```
trimmomatic-0.39.jar
```

Place it somewhere like:

```
/home/user/tools/Trimmomatic/
```

2. Suggested folder structure

```
project/
│
├── Trimmomatic/
│   └── trimmomatic-0.39.jar
│
├── data/
│   ├── 3_wildtype_S29_L004_R1.fastq
│   ├── 3_wildtype_S29_L004_R2.fastq
│
├── adapters/
│   └── 3_wildtype_adpters.fasta
│
└── output/
```

3. FULL PC VERSION OF YOUR SCRIPT

Save this as:

```
run_trimmomatic_3_wt.sh
```

✔️ Linux / macOS version (bash script)

```
#!/bin/bash

# ============================
# TRIMMOMATIC RUN (PC VERSION)
# ============================

# Paths
TRIMMOMATIC_PATH="/home/user/project/Trimmomatic/trimmomatic-0.39.jar"
ADAPTER_FILE="/home/user/project/adapters/3_wildtype_adpters.fasta"
INPUT_DIR="/home/user/project/data"
OUTPUT_DIR="/home/user/project/output"

# Create output directory
mkdir -p "$OUTPUT_DIR"

LOG_FILE="$OUTPUT_DIR/trim_3_wt_log.txt"

echo "Starting Trimmomatic processing for sample 3_wt..." | tee "$LOG_FILE"

# Input files
R1_FILE="$INPUT_DIR/3_wildtype_S29_L004_R1.fastq"
R2_FILE="$INPUT_DIR/3_wildtype_S29_L004_R2.fastq"

# Output files
PAIRED_R1="$OUTPUT_DIR/3_wt_R1_paired.fastq"
UNPAIRED_R1="$OUTPUT_DIR/3_wt_R1_unpaired.fastq"
PAIRED_R2="$OUTPUT_DIR/3_wt_R2_paired.fastq"
UNPAIRED_R2="$OUTPUT_DIR/3_wt_R2_unpaired.fastq"

# Check input files
if [[ -f "$R1_FILE" && -f "$R2_FILE" ]]; then

    echo "R1: $R1_FILE" | tee -a "$LOG_FILE"
    echo "R2: $R2_FILE" | tee -a "$LOG_FILE"

    java -jar "$TRIMMOMATIC_PATH" PE -threads 2 \
        "$R1_FILE" "$R2_FILE" \
        "$PAIRED_R1" "$UNPAIRED_R1" \
        "$PAIRED_R2" "$UNPAIRED_R2" \
        ILLUMINACLIP:"$ADAPTER_FILE":2:30:10 \
        LEADING:3 TRAILING:3 \
        SLIDINGWINDOW:4:15 \
        MINLEN:50

    echo "Finished processing 3_wt" | tee -a "$LOG_FILE"

else
    echo "ERROR: Missing R1 or R2 file" | tee -a "$LOG_FILE"
fi

echo "Done." | tee -a "$LOG_FILE"
```

4. How to run it on your PC

Step 1: give permission (Linux/macOS)

```
chmod +x run_trimmomatic_3_wt.sh
```

Step 2: run script

```
./run_trimmomatic_3_wt.sh
```

If you're on Windows

You don't use SLURM or bash script. Instead run directly in CMD or PowerShell:

```
java -jar trimmomatic-0.39.jar PE -threads 2 `
R1.fastq R2.fastq `
R1_paired.fastq R1_unpaired.fastq `
R2_paired.fastq R2_unpaired.fastq `
ILLUMINACLIP:adapters.fasta:2:30:10 `
LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15 MINLEN:50
```

SLURM script template for running Trimmomatic function.

```
#!/bin/bash
#SBATCH --job-name=trimmomatic_job
#SBATCH --output=trimmomatic_%j.out
#SBATCH --error=trimmomatic_%j.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --time=04:00:00
#SBATCH --mem=16G

# ========================
# Load environment
# ========================
module load Java/21.0.2

# ========================
# Paths (EDIT THESE)
# ========================
TRIMMOMATIC_JAR="/path/to/trimmomatic-0.39.jar"
ADAPTERS="/path/to/adapters.fasta"

INPUT_DIR="/path/to/input_fastq"
OUTPUT_DIR="/path/to/output"

mkdir -p "$OUTPUT_DIR"

# ========================
# Input files (CHANGE SAMPLE NAME ONLY)
# ========================
SAMPLE="sample_name"

R1="${INPUT_DIR}/${SAMPLE}_R1.fastq"
R2="${INPUT_DIR}/${SAMPLE}_R2.fastq"

# ========================
# Output files
# ========================
P_R1="${OUTPUT_DIR}/${SAMPLE}_R1_paired.fastq"
U_R1="${OUTPUT_DIR}/${SAMPLE}_R1_unpaired.fastq"
P_R2="${OUTPUT_DIR}/${SAMPLE}_R2_paired.fastq"
U_R2="${OUTPUT_DIR}/${SAMPLE}_R2_unpaired.fastq"

LOG="${OUTPUT_DIR}/${SAMPLE}_trimming.log"

echo "Starting Trimmomatic for $SAMPLE" > "$LOG"

# ========================
# Check input files
# ========================
if [[ -f "$R1" && -f "$R2" ]]; then

    echo "Files found, running Trimmomatic..." | tee -a "$LOG"

    java -jar "$TRIMMOMATIC_JAR" PE -threads $SLURM_CPUS_PER_TASK \
        "$R1" "$R2" \
        "$P_R1" "$U_R1" \
        "$P_R2" "$U_R2" \
        ILLUMINACLIP:"$ADAPTERS":2:30:10 \
        LEADING:3 TRAILING:3 \
        SLIDINGWINDOW:4:15 \
        MINLEN:50

    echo "Trimming completed for $SAMPLE" | tee -a "$LOG"

else
    echo "ERROR: Input files missing for $SAMPLE" | tee -a "$LOG"
fi

echo "Job finished." >> "$LOG"
```

How to use it

1. Save file

```
nano run_trimmomatic.slurm
```

2. Submit job

```
sbatch run_trimmomatic.slurm
```

How to run multiple samples (important upgrade)

Instead of editing manually, you can use:

```
SAMPLE_LIST="sample1 sample2 sample3"
for SAMPLE in $SAMPLE_LIST
do
    sbatch run_trimmomatic.slurm
done
```

step-by-step guide to run your STAR genome indexing on a normal PC- (2–8 hours)

Step 1 — Install STAR on your PC

✔️ Linux (recommended)

```
sudo apt update
sudo apt install star
```

Check:

```
STAR --version
```

✔️ macOS (via Homebrew)

```
brew install star
```

✔️ If not available (manual install)

```
git clone https://github.com/alexdobin/STAR.git
cd STAR/source
make STAR
```

Then run using:

```
./STAR
```

3. Step 2 — Prepare folders

Create a clean structure:

```
mkdir -p ~/star_project/genome
mkdir -p ~/star_project/index
mkdir -p ~/star_project/logs
```

4. Step 3 — Put your input files

Move these into a folder:

```
genome.fa
annotation.gtf
```

Example:

```
~/star_project/genome/Danio_rerio.fa
~/star_project/genome/Danio_rerio.gtf
```

Step 4 — Compute sjdbOverhang correctly

You used:

```
--sjdbOverhang 99
```

This is correct **only if your reads are 100 bp**.

Rule:

```
sjdbOverhang = readLength - 1
```

Examples:

- 50 bp reads → 49
- 100 bp reads → 99
- 150 bp reads → 149

Step 5 — Run STAR indexing (PC version)

Run this in terminal:

```
STAR \
--runThreadN 16 \
--runMode genomeGenerate \
--genomeDir ~/star_project/index \
--genomeFastaFiles ~/star_project/genome/Danio_rerio.fa \
--sjdbGTFfile ~/star_project/genome/Danio_rerio.gtf \
--sjdbOverhang 99
```

Step 6 — Save output log (optional)

```
STAR \
--runThreadN 16 \
--runMode genomeGenerate \
--genomeDir ~/star_project/index \
--genomeFastaFiles ~/star_project/genome/Danio_rerio.fa \
--sjdbGTFfile ~/star_project/genome/Danio_rerio.gtf \
--sjdbOverhang 99 \
> star_index.log 2>&1
```

Step 7 — Check output

If successful, your index folder will contain:

```
~/star_project/index/
```

Files like:

- Genome
- SA
- SAindex
- chrLength.txt

9. Expected time

Depends on your PC:

System	Time
Laptop	2–8 hours
Desktop (16 cores)	30–120 min
HPC	10–30 min

SLURM script template for running STAR genome indexing on an HPC system.

```
#!/bin/bash
#SBATCH --job-name=star_genome_index
#SBATCH --output=star_index_%j.out
#SBATCH --error=star_index_%j.err
#SBATCH --cpus-per-task=16
#SBATCH --time=08:00:00
#SBATCH --mem=64G

# =========================
# Load STAR module
# =========================
module load STAR

# =========================
# User-defined paths (EDIT THESE)
# =========================
GENOME_DIR="/path/to/output_star_index"
FASTA_FILE="/path/to/genome.fa"
GTF_FILE="/path/to/annotation.gtf"

# =========================
# Read length setting (IMPORTANT)
# sjdbOverhang = readLength - 1
# Example: 100bp reads → 99
# =========================
SJDB_OVERHANG=99

# =========================
# Create output directory
# =========================
mkdir -p "$GENOME_DIR"

echo "Starting STAR genome indexing..."
echo "Genome directory: $GENOME_DIR"
echo "FASTA: $FASTA_FILE"
echo "GTF: $GTF_FILE"

# =========================
# Run STAR genome indexing
# =========================
STAR \
--runThreadN $SLURM_CPUS_PER_TASK \
--runMode genomeGenerate \
--genomeDir "$GENOME_DIR" \
--genomeFastaFiles "$FASTA_FILE" \
--sjdbGTFfile "$GTF_FILE" \
--sjdbOverhang $SJDB_OVERHANG

echo "STAR genome indexing completed successfully."
```

How to use it

1. Save script

```
nano star_index.slurm
```

2. Edit paths

Replace:

```
/path/to/genome.fa
/path/to/annotation.gtf
```

3. Submit job

```
sbatch star_index.slurm
```

4. Check output

```
ls -lh /path/to/output_star_index
```

PC workflow for your STAR alignment

STEP 1 — Install STAR

✔️ Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install star
```

Check:

```
STAR --version
```

✔️ macOS

```
brew install star
```

✔️ If manual install needed

```
git clone https://github.com/alexdobin/STAR.git
cd STAR/source
make STAR
```

STEP 2 — Create folder structure

```
mkdir -p ~/rna_seq_project/index
mkdir -p ~/rna_seq_project/trimmed_fastq
mkdir -p ~/rna_seq_project/alignment_results
```

STEP 3 — Put your files in place

You need:

Genome index (already built using STAR)

```
~/rna_seq_project/index
```

Trimmed FASTQ files:

```
*_R1_paired.fastq
*_R2_paired.fastq
```

Example:

```
sample1_R1_paired.fastq
sample1_R2_paired.fastq
```

STEP 4 — Set paths (IMPORTANT)

Edit this:

```
GENOME_DIR="/home/user/rna_seq_project/index"
INPUT_DIR="/home/user/rna_seq_project/trimmed_fastq"
OUTPUT_DIR="/home/user/rna_seq_project/alignment_results"
```

STEP 5 — RUN STAR ALIGNMENT (PC SCRIPT)

Copy this into a file:

```
nano run_star_alignment.sh
```

FULL SCRIPT (PC VERSION)

```
#!/bin/bash

# =========================
# STAR alignment (PC version)
# =========================

GENOME_DIR="/home/user/rna_seq_project/index"
INPUT_DIR="/home/user/rna_seq_project/trimmed_fastq"
OUTPUT_DIR="/home/user/rna_seq_project/alignment_results"

mkdir -p "$OUTPUT_DIR"

echo "Starting STAR alignment..."

for R1 in "$INPUT_DIR"/*_R1_paired.fastq
do
  BASE=$(basename "$R1" _R1_paired.fastq)
  R2="$INPUT_DIR/${BASE}_R2_paired.fastq"

  if [[ -f "$R2" ]]; then

    STAR \
    --genomeDir "$GENOME_DIR" \
    --readFilesIn "$R1" "$R2" \
    --runThreadN 16 \
    --outFileNamePrefix "$OUTPUT_DIR/${BASE}_" \
    --outSAMtype BAM SortedByCoordinate

    echo "Completed: $BASE"
  else
    echo "Missing R2 for $BASE"
  fi
done

echo "All alignments finished."
```

STEP 6 — Make executable

```
chmod +x run_star_alignment.sh
```

STEP 7 — RUN IT

```
./run_star_alignment.sh
```

STEP 8 — Output you will get

Inside:

```
alignment_results/
```

You will see:

```
sample1_Aligned.sortedByCoord.out.bam
sample1_Log.final.out
sample1_Log.out
sample1_Log.progress.out
```

SLURM script for running STAR alignment for multiple samples

```
#!/bin/bash
#SBATCH --job-name=star_align
#SBATCH --output=star_align_%j.out
#SBATCH --error=star_align_%j.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --time=08:00:00
#SBATCH --mem=64G

# ========================
# Load STAR module
# ========================
module load STAR

# ========================
# USER PATHS (EDIT THESE)
# ========================
GENOME_DIR="/path/to/star_genome_index"
INPUT_DIR="/path/to/trimmed_fastq"
OUTPUT_DIR="/path/to/alignment_results"

mkdir -p "$OUTPUT_DIR"

echo "Starting STAR alignment pipeline..."

# ========================
# LOOP OVER ALL SAMPLES
# ========================
for R1 in "$INPUT_DIR"/*_R1_paired.fastq
do
    BASE=$(basename "$R1" _R1_paired.fastq)
    R2="$INPUT_DIR/${BASE}_R2_paired.fastq"

    if [[ -f "$R2" ]]; then

        STAR \
        --genomeDir "$GENOME_DIR" \
        --readFilesIn "$R1" "$R2" \
        --runThreadN $SLURM_CPUS_PER_TASK \
        --outFileNamePrefix "$OUTPUT_DIR/${BASE}_" \
        --outSAMtype BAM SortedByCoordinate

        echo "Completed: $BASE"

    else
        echo "Missing R2 file for $BASE"
    fi
done

echo "STAR alignment finished for all samples."
```

HOW TO USE

1. Save file

```
nano star_align.slurm
```

2. Edit paths

Replace:

```
/path/to/...
```

3. Submit job

```
sbatch star_align.slurm
```

Here is the complete step-by-step guide to run your HTSeq-count pipeline on a PC (no SLURM) from scratch.

This is based on:

HTSeq

STEP 1 — Install HTSeq

✔️ Install via pip (recommended)

```
pip install htseq
```

Check:

```
htseq-count --version
```

✔️ If conda available

```
conda install -c bioconda htseq
```

STEP 2 — Prepare folders

```
mkdir -p ~/htseq_project/bam
mkdir -p ~/htseq_project/output
mkdir -p ~/htseq_project/temp
```

STEP 3 — Put your files

You need:

✔️ BAM files (from STAR)

```
sample1.bam
sample2.bam
sample3.bam
```

✔️ GTF file

```
Danio_rerio.gtf
```

STEP 4 — Set paths

Edit:

```
BAM_DIR=~/htseq_project/bam
GTF_FILE=~/htseq_project/Danio_rerio.gtf
OUTPUT_DIR=~/htseq_project/output
```

STEP 5 — CREATE SCRIPT FILE

```
nano run_htseq.sh
```

FULL PC VERSION SCRIPT

Copy this:

```
#!/bin/bash

# ==========================
# HTSeq-count pipeline (PC version)
# ==========================

BAM_DIR="$HOME/htseq_project/bam"
GTF_FILE="$HOME/htseq_project/Danio_rerio.gtf"
OUTPUT_DIR="$HOME/htseq_project/output"

mkdir -p "$OUTPUT_DIR/temp"

MATRIX_TMP="$OUTPUT_DIR/matrix.tmp"
MATRIX_FILE="$OUTPUT_DIR/combined_counts_matrix.txt"

FIRST=1

echo "Starting HTSeq-count pipeline..."

for BAM_FILE in "$BAM_DIR"/*.bam
do
  SAMPLE=$(basename "$BAM_FILE" .bam)
  TEMP_FILE="$OUTPUT_DIR/temp/${SAMPLE}.txt"

  htseq-count -f bam -r pos -s no -i gene_id \
  "$BAM_FILE" "$GTF_FILE" > "$TEMP_FILE"

  grep -v '^__' "$TEMP_FILE" > "${TEMP_FILE}.clean"
  mv "${TEMP_FILE}.clean" "$TEMP_FILE"

  if [ $FIRST -eq 1 ]; then
    cut -f1,2 "$TEMP_FILE" > "$MATRIX_TMP"
    FIRST=0
  else
    cut -f2 "$TEMP_FILE" > "$OUTPUT_DIR/temp/${SAMPLE}_counts.txt"
    paste "$MATRIX_TMP" "$OUTPUT_DIR/temp/${SAMPLE}_counts.txt" > "$OUTPUT_DIR/temp/tmp_matrix.txt"
    mv "$OUTPUT_DIR/temp/tmp_matrix.txt" "$MATRIX_TMP"
  fi

  echo "Finished $SAMPLE"
done

# Create header
HEADER="Gene_ID"

for BAM_FILE in "$BAM_DIR"/*.bam
do
  SAMPLE=$(basename "$BAM_FILE" .bam)
  HEADER="${HEADER}\t${SAMPLE}"
done

echo -e "$HEADER" > "$MATRIX_FILE"
cat "$MATRIX_TMP" >> "$MATRIX_FILE"

rm -r "$OUTPUT_DIR/temp" "$MATRIX_TMP"

echo "DONE: Matrix saved at $MATRIX_FILE"
```

STEP 6 — Make executable

```
chmod +x run_htseq.sh
```

STEP 7 — RUN IT

```
./run_htseq.sh
```

10. OUTPUT YOU WILL GET

```
combined_counts_matrix.txt
```

Example:

```
Gene_ID   sample1   sample2   sample3
geneA     10        5         8
geneB     0         2         1
```

SLURM script for running HTSeq-count + matrix generation pipeline on HPC.

```
#!/bin/bash
#SBATCH --job-name=htseq_count
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=08:00:00
#SBATCH --output=htseq_%j.out
#SBATCH --error=htseq_%j.err
#SBATCH --ntasks=1

# ==========================
# Load HTSeq module
# ==========================
module load HTSeq

# ==========================
# USER INPUT PATHS (EDIT THESE)
# ==========================
BAM_DIR="/path/to/bam_files"
GTF_FILE="/path/to/annotation.gtf"
OUTPUT_DIR="/path/to/htseq_output"

# ==========================
# CREATE DIRECTORIES
# ==========================
mkdir -p "$OUTPUT_DIR/temp"

MATRIX_TMP="$OUTPUT_DIR/matrix.tmp"
MATRIX_FILE="$OUTPUT_DIR/combined_counts_matrix.txt"

FIRST=1

echo "Starting HTSeq-count pipeline..."

# ==========================
# LOOP OVER BAM FILES
# ==========================
for BAM_FILE in "$BAM_DIR"/*.bam
do
   SAMPLE=$(basename "$BAM_FILE" .bam)
   TEMP_FILE="$OUTPUT_DIR/temp/${SAMPLE}.txt"

   # Run HTSeq-count
   htseq-count -f bam -r pos -s no -i gene_id \
   "$BAM_FILE" "$GTF_FILE" > "$TEMP_FILE"

   # Remove summary rows
   grep -v '^__' "$TEMP_FILE" > "${TEMP_FILE}.clean"
   mv "${TEMP_FILE}.clean" "$TEMP_FILE"

   # Build matrix
   if [ $FIRST -eq 1 ]; then
      cut -f1,2 "$TEMP_FILE" > "$MATRIX_TMP"
      FIRST=0
   else
      cut -f2 "$TEMP_FILE" > "$OUTPUT_DIR/temp/${SAMPLE}_counts.txt"
      paste "$MATRIX_TMP" "$OUTPUT_DIR/temp/${SAMPLE}_counts.txt" > "$OUTPUT_DIR/temp/tmp_matrix.txt"
      mv "$OUTPUT_DIR/temp/tmp_matrix.txt" "$MATRIX_TMP"
   fi

   echo "Completed: $SAMPLE"
done

# ==========================
# CREATE HEADER
# ==========================
HEADER="Gene_ID"

for BAM_FILE in "$BAM_DIR"/*.bam
do
   SAMPLE=$(basename "$BAM_FILE" .bam)
   HEADER="${HEADER}\t${SAMPLE}"
done

echo -e "$HEADER" > "$MATRIX_FILE"
cat "$MATRIX_TMP" >> "$MATRIX_FILE"

# ==========================
# CLEANUP
# ==========================
rm -r "$OUTPUT_DIR/temp" "$MATRIX_TMP"

echo "HTSeq-count completed. Matrix saved at:"
echo "$MATRIX_FILE"
```

HOW TO USE

1. Save script

```
nano htseq.slurm
```

2. Edit paths

Replace:

```
/path/to/bam_files
/path/to/annotation.gtf
```

3. Submit job

```
sbatch htseq.slurm
```

Differential Expression Analysis using DESeq2 step-by-step workflow :

STEP 1 — Install R + Required Libraries

✔️ Install R

Download: https://cran.r-project.org

✔️ Install RStudio (recommended)

https://posit.co/download/rstudio-desktop/

✔️ Install DESeq2 packages

Open R / RStudio:

```
if (!requireNamespace("BiocManager", quietly = TRUE))
    install.packages("BiocManager")

BiocManager::install("DESeq2")
BiocManager::install("apeglm")
install.packages("ggplot2")
install.packages("pheatmap")
```

STEP 2 — Load libraries (your Step 1)

```
library(DESeq2)
library(ggplot2)
library(pheatmap)
```

STEP 3 — Read count matrix (your Step 2)

```
counts <- read.table("combined_counts_matrix.txt",
                     header = TRUE,
                     row.names = 1,
                     sep = "\t",
                     check.names = FALSE)

head(counts)
```

STEP 4 — Sample metadata (your Step 3)

You must manually create metadata:

```
sampleTable <- data.frame(
  row.names = colnames(counts),
  condition = c("WT", "WT", "KO", "KO")   # CHANGE THIS
)

sampleTable
```

STEP 5 — Check sample order (your Step 4)

VERY IMPORTANT:

```
all(rownames(sampleTable) == colnames(counts))
```

If FALSE:

```
counts <- counts[, rownames(sampleTable)]
```

STEP 6 — Set reference (WT baseline) (your Step 5)

```
sampleTable$condition <- relevel(factor(sampleTable$condition), ref = "WT")
```

STEP 7 — Build DESeq2 object (your Step 6)

```
dds <- DESeqDataSetFromMatrix(
  countData = counts,
  colData = sampleTable,
  design = ~ condition
)
```

STEP 8 — Filter low counts

```
dds <- dds[rowSums(counts(dds)) > 10, ]
```

STEP 9 — Run DESeq2 analysis

```
dds <- DESeq(dds)
```

STEP 10 — Get results

```
res <- results(dds)
res <- lfcShrink(dds, coef=2, type="apeglm")

head(res)
```

STEP 11 — Save results

```
write.csv(as.data.frame(res),
          file = "DESeq2_results.csv")
```

STEP 12 — Visualization

MA Plot

```
plotMA(res)
```

PCA plot

```
vsd <- vst(dds, blind = FALSE)
plotPCA(vsd, intgroup = "condition")
```

Heatmap

```
topGenes <- head(order(res$padj), 50)
mat <- assay(vsd)[topGenes, ]

pheatmap(mat, scale = "row")
```

Here is a **complete GO (Gene Ontology) enrichment analysis code in R** for your DESeq2 results.

This works directly after you have:

```
res <- results(dds)
```

We will use:

clusterProfiler

and GO database annotation.

STEP 1 — Install required packages (run once)

```
if (!requireNamespace("BiocManager", quietly = TRUE))
    install.packages("BiocManager")

BiocManager::install("clusterProfiler")
BiocManager::install("org.Dr.eg.db")  # zebrafish database (change if needed)
BiocManager::install("enrichplot")
install.packages("ggplot2")
```

STEP 2 — Load libraries

```
library(clusterProfiler)
library(org.Dr.eg.db)
library(enrichplot)
library(ggplot2)
```

STEP 3 — Prepare DE gene list

Filter significant genes

```
sig <- res[which(res$padj < 0.05), ]
sig <- sig[!is.na(sig$padj), ]
```

Convert gene IDs (IMPORTANT)

If your rownames are ENSEMBL IDs:

```
gene_list <- rownames(sig)
```

STEP 4 — Convert IDs to Entrez IDs

```
gene_convert <- bitr(gene_list,
                     fromType = "ENSEMBL",
                     toType = "ENTREZID",
                     OrgDb = org.Dr.eg.db)
```

STEP 5 — GO enrichment analysis

Biological Process (BP)

```
go_bp <- enrichGO(gene         = gene_convert$ENTREZID,
                  OrgDb        = org.Dr.eg.db,
                  keyType      = "ENTREZID",
                  ont          = "BP",
                  pAdjustMethod = "BH",
                  pvalueCutoff = 0.05,
                  qvalueCutoff = 0.05)
```

Molecular Function (MF)

```
go_mf <- enrichGO(gene_convert$ENTREZID,
                  OrgDb = org.Dr.eg.db,
                  keyType = "ENTREZID",
                  ont = "MF",
                  pAdjustMethod = "BH",
                  pvalueCutoff = 0.05)
```

Cellular Component (CC)

```
go_cc <- enrichGO(gene_convert$ENTREZID,
                  OrgDb = org.Dr.eg.db,
                  keyType = "ENTREZID",
                  ont = "CC",
                  pAdjustMethod = "BH",
                  pvalueCutoff = 0.05)
```

STEP 6 — Visualization

Dotplot (most important)

```
dotplot(go_bp, showCategory = 20) + ggtitle("GO Biological Process")
```

Barplot

```
barplot(go_bp, showCategory = 20)
```

Network plot

```
cnetplot(go_bp, showCategory = 10)
```

Combine all GO

```
library(enrichplot)
emapplot(pairwise_termsim(go_bp))
```

STEP 7 — Save results

```
write.csv(as.data.frame(go_bp), "GO_BP_results.csv")
write.csv(as.data.frame(go_mf), "GO_MF_results.csv")
write.csv(as.data.frame(go_cc), "GO_CC_results.csv")
```

R workflow that does exactly what gens you want:
Neural GO terms → Neural genes
STEP 0 — Install packages (run once)

```
install.packages("ontologyIndex")

if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

BiocManager::install(c(
  "clusterProfiler",
  "org.Hs.eg.db"
))
```

STEP 1 — Download GO ontology file

```
url <- "http://current.geneontology.org/ontology/go-basic.obo"
destfile <- "go-basic.obo"

download.file(url, destfile, method = "auto")
```

STEP 2 — Load GO ontology

```
library(ontologyIndex)

go <- get_ontology(destfile, extract_tags = "minimal")
```

STEP 3 — Define function to search GO terms (multi-keyword)

```
get_go_terms_multiword <- function(go, keywords) {

  pattern <- paste(keywords, collapse = "|")

  idx <- grepl(pattern, go$name, ignore.case = TRUE)

  if (!is.null(go$synonym)) {
    idx <- idx | grepl(pattern, go$synonym, ignore.case = TRUE)
  }

  data.frame(
    go_id = go$id[idx],
    name  = go$name[idx],
    stringsAsFactors = FALSE
  )
}
```

STEP 4 — Define neural-related keywords

You can edit this list anytime:

```
neural_keywords <- c(
  "neuron",
  "neuronal",
  "neural",
  "synapse",
  "axon",
  "dendrite",
  "brain",
  "nervous",
  "gli",
  "cortex",
  "hippocampus",
  "habenula"
)
```

STEP 5 — Extract neural GO terms

```
neural_terms <- get_go_terms_multiword(go, neural_keywords)

head(neural_terms)
```

STEP 6 — Map GO terms → genes

```
library(clusterProfiler)
library(org.Hs.eg.db)

go_ids <- unique(neural_terms$go_id)
```

Convert GO → Entrez genes

```
gene_list <- lapply(go_ids, function(goid) {
  tryCatch({
    bitr_go(
      goid,
      OrgDb = org.Hs.eg.db,
      keyType = "GOALL",
      column = "ENTREZID"
    )
  }, error = function(e) NULL)
})
```

STEP 7 — Combine all neural genes

```
all_genes <- do.call(rbind, gene_list)

neural_genes <- unique(all_genes$ENTREZID)

length(neural_genes)
head(neural_genes)
```

STEP 8 — Save outputs

GO terms

```
write.table(
  neural_terms,
  "neural_GO_terms.txt",
  sep = "\t",
  row.names = FALSE,
  quote = FALSE
)
```

Gene list

```
write.table(
  neural_genes,
  "neural_gene_list.txt",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)
```

SLURM pipeline script for RNA-seq differential expression workflow + GO enrichment analysis

```
#!/bin/bash
#SBATCH --job-name=RNAseq_DE_GO
#SBATCH --output=RNAseq_DE_GO_%j.out
#SBATCH --error=RNAseq_DE_GO_%j.err
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=12:00:00
#SBATCH --ntasks=1

# ============================
# Load R module
# ============================
module load R

# ============================
# USER INPUT PATHS (EDIT THESE)
# ============================
COUNT_MATRIX="/path/to/combined_counts_matrix.txt"
OUTPUT_DIR="/path/to/output_DE_GO"
mkdir -p "$OUTPUT_DIR"

# ============================
# RUN R SCRIPT INLINE
# ============================
Rscript - <<EOF

# ============================
# LOAD LIBRARIES
# ============================
library(DESeq2)
library(clusterProfiler)
library(enrichplot)
library(ggplot2)
library(org.Dr.eg.db)   # CHANGE if organism differs

# ============================
# READ COUNT MATRIX
# ============================
counts <- read.table("$COUNT_MATRIX",
                     header = TRUE,
                     row.names = 1,
                     sep = "\t",
                     check.names = FALSE)

# ============================
# SAMPLE METADATA (EDIT THIS)
# ============================
sampleTable <- data.frame(
  row.names = colnames(counts),
  condition = c("WT","WT","KO","KO")   # EDIT
)

# ensure correct order
counts <- counts[, rownames(sampleTable)]

# ============================
# DESEQ2 ANALYSIS
# ============================
dds <- DESeqDataSetFromMatrix(counts, sampleTable, design = ~ condition)
dds <- dds[rowSums(counts(dds)) > 10, ]
dds <- DESeq(dds)

res <- results(dds)
res <- res[!is.na(res$padj), ]

write.csv(as.data.frame(res),
          file = "$OUTPUT_DIR/DESeq2_results.csv")

# ============================
# SIGNIFICANT GENES
# ============================
sig <- res[which(res$padj < 0.05), ]
genes <- rownames(sig)

# ============================
# GENE ID CONVERSION
# ============================
gene_df <- bitr(genes,
                fromType="ENSEMBL",
                toType="ENTREZID",
                OrgDb=org.Dr.eg.db)

# ============================
# GO ENRICHMENT (BP)
# ============================
go_bp <- enrichGO(gene = gene_df$ENTREZID,
                  OrgDb = org.Dr.eg.db,
                  keyType = "ENTREZID",
                  ont = "BP",
                  pAdjustMethod = "BH",
                  pvalueCutoff = 0.05)

# ============================
# SAVE GO RESULTS
# ============================
write.csv(as.data.frame(go_bp),
          file = "$OUTPUT_DIR/GO_BP_results.csv")

# ============================
# PLOTS
# ============================
png("$OUTPUT_DIR/GO_dotplot.png", width=1000, height=800)
dotplot(go_bp, showCategory=20)
dev.off()

png("$OUTPUT_DIR/GO_barplot.png", width=1000, height=800)
barplot(go_bp, showCategory=20)
dev.off()

# ============================
# DONE
# ============================
print("DESeq2 + GO analysis completed successfully")
EOF
```

HOW TO RUN

1. Save script

```
nano rnaseq_pipeline.slurm
```

2. Edit these two things:

```
COUNT_MATRIX
condition metadata
```

3. Submit job

```
sbatch rnaseq_pipeline.slurm
```

filtering DESeq2 significant genes.

- `neural_genes` → from GO terms (Entrez IDs)
- DESeq2 results → differential expression out

STEP 1 — Load DESeq2 results

Assume you already ran DESeq2:

```
library(DESeq2)

res <- results(dds)
```

Make it a clean table

```
res_df <- as.data.frame(res)
res_df$gene_id <- rownames(res_df)
```

IMPORTANT CHECK (VERY COMMON ISSUE)

Your GO genes are likely:

- Entrez IDs (`1234`, `5678`)

But DESeq2 genes are often:

- Ensembl IDs (`ENSG...`)
- or SYMBOLS (`GAP43`, etc.)

👉 You MUST match ID types first.

STEP 2 — Convert DESeq2 IDs to Entrez (if needed)

If your DESeq2 uses gene SYMBOLS:

```
library(org.Hs.eg.db)
library(clusterProfiler)

conv <- bitr(
  res_df$gene_id,
  fromType = "SYMBOL",
  toType   = "ENTREZID",
  OrgDb    = org.Hs.eg.db
)
```

Merge with DESeq2 results

```
res_merged <- merge(res_df, conv, by.x = "gene_id", by.y = "SYMBOL")
```

Now you have:

- DESeq2 stats
- ENTREZID (matching GO list)

STEP 3 — Define DESeq2 significant genes

Example thresholds:

```
sig <- res_merged[
  res_merged$padj < 0.05 &
  abs(res_merged$log2FoldChange) > 1,
]
```

STEP 4 — Intersect with GO neural genes

Now your GO genes:

```
neural_genes   # from previous step (ENTREZ IDs)
```

Filter:

```
neural_sig <- sig[sig$ENTREZID %in% neural_genes, ]
```

RESULT

```
head(neural_sig)
```

You now have:

✔️ significantly DE genes ✔️ that are neural GO-related

STEP 5 — Save final gene list

```
write.table(
  neural_sig,
  "neural_DEG_filtered.txt",
  sep = "\t",
  row.names = FALSE,
  quote = FALSE
)
```

OPTIONAL (STRONGLY RECOMMENDED)

1. Split up/down regulated neural genes

```
up_neural <- neural_sig[neural_sig$log2FoldChange > 1, ]
down_neural <- neural_sig[neural_sig$log2FoldChange < -1, ]
```

2. Quick summary

```
cat("Total neural DE genes:", nrow(neural_sig), "\n")
cat("Upregulated:", nrow(up_neural), "\n")
cat("Downregulated:", nrow(down_neural), "\n")
```

3. Volcano highlight (optional)

```
plot(res$log2FoldChange, -log10(res$padj))
points(neural_sig$log2FoldChange, -log10(neural_sig$padj), col="red", pch=16)
```